HKU Drone Club

Deployment and Verification Guide

CPT + Campus-Memo · COMP7610 Smart Contract Lab

Full Name · UID · Sepolia Chain ID 11155111

1. Purpose and design

Follow the steps in order, then compare the observed chain state with the checklist. CPT is the club's fungible economic resource; Campus-Memo is a scarce commemorative record. A member's exit burns remaining CPT and revokes future club privileges, while an already-earned NFT remains with its owner.

CPT — club points

Abstract Data Type

What it represents:
The constructor mints exactly 1,000 CPT to the token contract as treasury reserve. Only `ADMIN_ROLE` can mint, distribute treasury points, or call `adminBurnPoints()` to permanently deduct a specified amount from a user's balance.

Implementation methods:
- `applyForMembership()` → application.
- `approveMembership()` → membership + 50 welcome CPT.
- `withdrawFromClub()` → burn remaining CPT + revoke membership.

Main operations / functions:
- Standard `transfer`, `approve`, and `transferFrom` remain available.
- Admin-only `adminRecoverPoints(account, amount)` returns CPT to treasury without user approval; supply stays constant.
- Withdrawal does not touch existing NFTs.

Campus-Memo — limited NFT

Abstract Data Type

What it represents:
Only CPT administrators call `mintMemo(memberAddress)` to issue an NFT to a member. The admin approves and pays 100 CPT, which is burned. Lifetime cap: 10; no NFT burn or admin recovery.

Implementation methods:
- Token IDs start at 1; URI is IPFS base URI + ID.
- `safeTransferFrom()` supports NFT transfers.

Main operations / functions:
- Admin role and recipient membership are checked; the member pays no CPT.
- Existing NFTs remain collectible after withdrawal.
- `listMemo()` + `buyMemo()` exchange an escrowed NFT for CPT; `cancelMemoListing()` returns it.

Run the local verification first:

```
npm install
npm run compile
npm test
```

2. Sepolia operation

Step 1 — Prepare wallet
操作： Select Sepolia in MetaMask and obtain test ETH. Use a test-only account.
预期： MetaMask shows Chain ID `11155111` and enough ETH for deployments and calls.
为什么： *The report needs public, inspectable Sepolia addresses.*

Step 2 — Prepare URI
操作： Pin ten JSON metadata files to IPFS and set `ipfs://CID/` as the base URI.
预期： `ipfs://CID/1.json` through `10.json` are reachable and include name, description, image, and attributes.
为什么： *The URI is part of the NFT identity and must be reported.*

Step 3 — Deploy CPT
操作： In Remix compile `contracts/CPT.sol` with Solidity 0.8.24, choose `Injected Provider – MetaMask`, and deploy `CPT(initialAdmin)`.
预期： Record the address; `totalSupply()` and treasury `balanceOf` are both 1,000 CPT.
为什么： *The first batch is an auditable treasury reserve.*

Step 4 — Deploy Campus-Memo
操作： Deploy `CampusMemo(cptAddress, "ipfs://CID/")` with the CPT address from Step 3.
预期： Record the address; `MAX_SUPPLY()` is 10 and `MINT_COST()` is 100 CPT.
为什么： *The NFT checks the admin role and recipient membership; the admin pays the mint cost.*

Step 5 — Onboard a member
操作： From a student account call `applyForMembership()`. As admin call `approveMembership(student)`.
预期： `isMember(student)` is true and the balance increases by 50 CPT; a non-admin approval reverts.
为什么： *Members request access, but administrators alone distribute treasury points.*

Step 6 — Mint, transfer, and trade
操作： The admin obtains 100 CPT, calls CPT `approve(campusMemoAddress, 100000000000000000000)`, then NFT `mintMemo(memberAddress)`. Test CPT `transfer` and NFT `safeTransferFrom`. An NFT owner can call `listMemo(tokenId, price)`; a buyer approves that CPT amount and calls `buyMemo(tokenId)`.
预期： Token 1 is owned by the member, a purchase pays CPT to the seller and changes `ownerOf(1)`, and `cancelMemoListing()` returns an unsold token.
为什么： *CPT is fungible; each NFT is individually identified, transferable, and tradeable after escrow.*

Step 7 — Exit without losing the souvenir
操作： The member calls `withdrawFromClub()`.
预期： CPT balance is zero, `isMember=false`, `MemberWithdrawn` is emitted, and an unsold `ownerOf(1)` is still the former member.
为什么： *Membership status controls future privileges and does not confiscate a commemorative NFT.*

Step 8 — Recover points for reuse
操作： As admin call `adminRecoverPoints(user, 2000000000000000000)` to recover 20 CPT from a funded user.
预期： User balance falls by 20 CPT; treasury balance rises by 20; supply, membership, and NFTs stay unchanged. Check `PointsRecoveredByAdmin`; non-admin calls revert.
为什么： *Recovery reuses existing points; `adminBurnPoints()` permanently destroys them.*

3. Verification checklist

Check	Expected	Evidence
Initial supply	1,000 CPT	`totalSupply()` + treasury `balanceOf`
Welcome allocation	50 CPT	`MembershipApproved` + balance
NFT issuance	Admin only; member recipient	Member mint reverts; `MemoMinted(admin, recipient, id)`
NFT payment	100 admin CPT burned	`totalSupply()` before/after mint
Scarcity	No token 11	reverted 11th mint
Exit policy	CPT burned; NFT retained	`isMember=false`, zero CPT, unchanged `ownerOf(1)`
Metadata	Correct URI	`tokenURI(1)` + IPFS check

4. Submission record

- CPT: `REPLACE_WITH_SEPOLIA_ADDRESS` · Campus-Memo: `REPLACE_WITH_SEPOLIA_ADDRESS`
- Transactions: `REPLACE_WITH_TX_HASHES`
- NFT URI: `ipfs://REPLACE_WITH_CID/` · `tokenURI(1)`: `REPLACE_WITH_TOKEN_URI`
- Student: `REPLACE_WITH_FULL_NAME_AND_UID`

Insert screenshots of MetaMask, both deployments, membership approval, NFT mint/transfer/purchase, and withdrawal with unchanged `ownerOf(1)` when unsold. This compact Typst version is additional; the existing LaTeX report is retained alongside it.